Project Acceptance Tests

Clan Management Platform - Sprint 1 Deliverable

Table of Contents

1. Objectives
2. Document References
3. Test Summary
4. Testing Strategy
5. Test A
6. Test B
7. Test C
8. Test D
9. Test E
10. Test F
11. Testing Schedule
12. Testing Materials

Objectives

The purpose of this document is to define the acceptance tests for the CLAN Management Platform. The tests will verify the User Authentication, Clan Creation and Tag Validation, Membership Management, invitation workflow, administration and moderation and the REST API. Testing will include manual system testing, unit testing, API testing, and integration testing.

Document References

This test manual refers to the following project documents:

- *Client project requirements specification*
- *Scope of work*
- *Sprint2 user story*

The Client project requirements specification and Scope of work define the features and functions to be tested. The user stories describe how these requirements will be implemented. This test manual defines the procedures used to determine whether the implementation satisfies the requirements.

Test Summary

This document covers 6 major areas of the CLAN Management Platform:

- *Test A - User Authentication (OIDC login): Verifies authentication through an external OpenID Connect provider (OIDC), session establishment, protected-route enforcement and logout.*
- *Test B - Clan Creation and Tag Validation: Verifies clan creation and enforcement of clan tag rules, including uniqueness, case-insensitivity, max length, allowed characters.*
- *Test C - Membership Management: Verifies joining and leaving clans, member removal, ownership transfer and enforcement of the 'one active clan membership rule'.*
- *Test D - Invitation Workflow: Verifies the invitation lifecycle, including creation, acceptance, decline, revocation, expiration and invitation status tracking.*
- Test E - Administration and Moderation: Verifies administrator capabilities to list, search, suspend, rename, modify invalid tags on, and delete clans, plus the report queue, moderation history, and access control restricting these actions to admins.
- Test F - REST API - Verifies the core clan REST API endpoints, request/response schemas, status codes and error handling.

Testing Strategy

Subsystems to be tested

The main subsystems covered by the acceptance tests are:

- *User authentication*
- *Clan creation and tag validation*
- *Membership management*
- *Invitation workflow*
- *Administration and moderation*
- *REST API*

System integration strategy

The tests will verify interactions between the web interface, application backend, REST API, PostgreSQL database and external OIDC provider. Integration testing will focus on whether data transfer and user actions are handled correctly across the components.

How the tests will be conducted

Testing will include manual testing through the web interface, direct and automated REST API testing, unit testing for validation and business logic. And integration testing between the application, PostgreSQL database, REST API and OIDC provider.

Where the test will be conducted

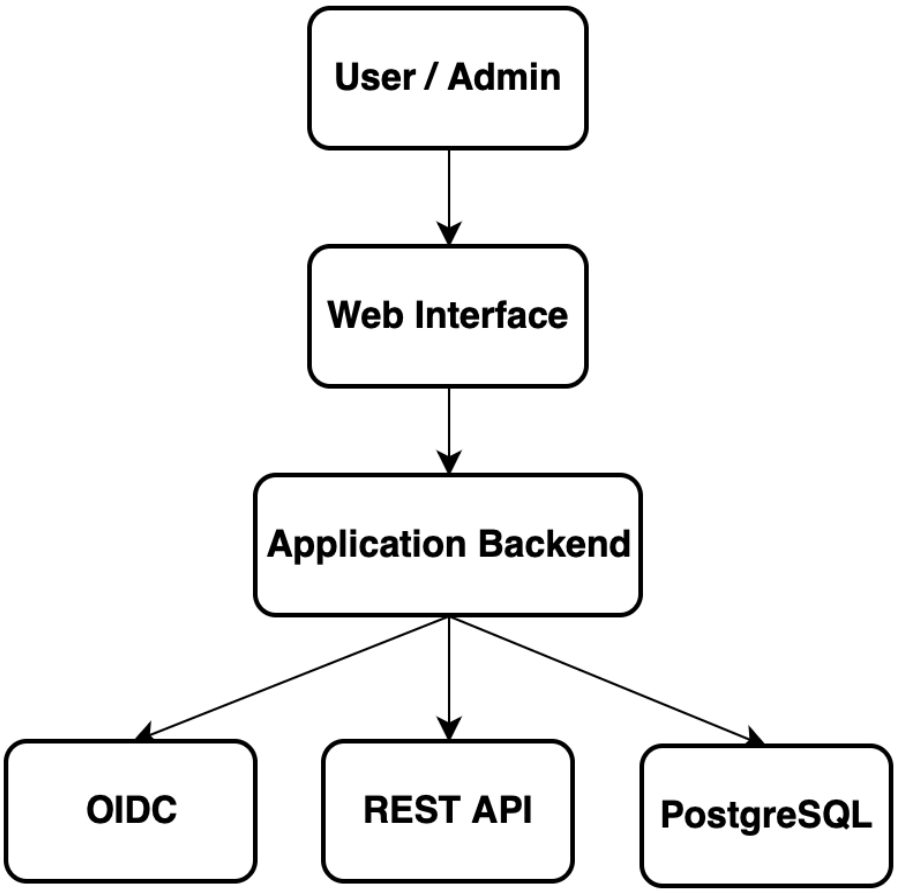
Tests will be carried out in the team's local environment using the deployed Clan Management Platform.

When the test will be conducted

During Sprint 2 and Sprint 3 after the relevant functionality has been implemented.

By whom

Tests will be carried out by team members listed in the testing schedule below. The diagram below shows the main system components involved in testing.


User / Admin
Web Interface
Application Backend
OIDC
REST API
PostgreSQL

Test A : User Authentication (OIDC login)

Verifies that a user can authenticate via an external OpenID connect provider, that a session is correctly established, and that unauthenticated users cannot access protected resources.

Test Specification

Demonstrates satisfaction of the client's Authentication requirements: the application authenticates via an external OIDC provider and must not implement local password management.

Method tested:

- *Login redirect flow*
- *Session establishment*
- *protected - route enforcement*
- *Logout*

Test Description

Location of test: Web UI login page, deployed to the local environment

Means of Control: Manual via browser interaction using a test identity at the OIDC provider

Data:

- Input Data: Valid test user credentials registered with the OIDC provider
- Input Commands: Navigate to a protected page while logged out; initiate login; submit idP credentials; logout
- Output Data: Authenticated session/token; user redirected to their account page after login
- System Messages: Redirected to IdP for unauthenticated access attempts; confirmation of successful login/logout

Procedures:

1. Navigate to the application without an active session; confirm protected routes redirect to login
2. Initiate login; confirm redirect to the external OIDC provider
3. Authenticate with valid test credentials at the provider
4. Confirm redirect back to the application with an active session and visible account info
5. Log out and confirm the session is invalidated and protected routes again redirect to login

Test Analysis Report

To be completed following test execution in Sprint 2/3 . Planned analysis criteria:

- **Function:** Confirm whether OIDC login, session handling, and logout function
- **Performance:** Record login round-trip time (redirect to authenticated state)
- **Data measures:** Pass or Fail recorded against each of the 5 procedure steps
- **Errors/Deficiencies:** Any authentication failure is high severity, since all other functionality is gated behind it

---

Test B: Clan Creation & Tag validation

Verifies that clan creation succeeds only with a valid tag, and that all clan tag rules are enforced

Test Specification

Demonstrated satisfaction of the Client's Clan Tag Rules and the 'create a clan' user feature.

Method tested:

- *Clan creation form/endpoint*
- *Tag validation logic (Uniqueness, case-insensitivity, max length, allowed characters)*

Test Description

Location of test: Web UI 'Create Clan' form, and POST /clans API endpoint

Means of Control: Manual via the UI, supplemented by direct API calls for edge cases

Data:

- Input Data: A valid 3 character alphanumeric tag; together with a clan name , description, and visibility setting (Public/Private); a duplicate tag differing only in case; tag exceeding 3 character; a tag containing non alphanumeric characters
- Input Commands: Submit create clan form / POST/ clans with each input above
- Output Data: A created clan record for the valid case, with name, tag, description, owner, visibility, and creation timestamp all stored correctly ; a validation error for each invalid case
- System Messages: Success confirmation; specific validation error messages identifying which rule was violated

Procedures:

1. Create a clan with a valid 3 character alphanumeric tag; confirm success
2. Attempt to create a second clan with the same tag in different case; confirm rejection (case insensitive uniqueness)
3. Attempt a tag longer than 3 character ; confirm rejection
4. Attempt a tag containing non-alphanumeric character; confirm rejection
5. Create a clan supplying name, desc, and visibility alongside the tag; confirm all fields are saved and returned correctly (not just the tag)
6. Confirm the successfully created clan appears correctly in clan search and listing

Test Analysis Report

To be completed following test execution in Sprint 2/3 . Planned analysis criteria:

- **Function:** Confirm clan creation and all four tag validation rule
- **Performance:** Note response time for validation errors
- **Data measures:** Pass/Fail per validation rule, plus the success case (6 checks in total)
- **Errors/Deficiencies:** Any unenforced rule risks the database uniqueness constraint and should be logged as high severity

---

Test C: Membership Management

Verifies join, leave, remove members, transfer ownership operations, and enforcement of the 'exactly one active clan membership per user' rule and the distinct join paths for Public and Private clans.

Test Specification

Demonstrates satisfaction of the client's Membership Management requirements, including the supported operations (join, leave, remove member, transfer ownership) , the hard constraint that each user may hold at most one active membership, and the Clan visibility rules governing how a user may join a Public Clan (request to join) and a Private clan(invitation only)

Test Description

Location of test: Web UI clan management pages and the corresponding REST endpoint (join, leave, member removal, ownership transfer)

Means of Control: Manual, using two or more test user accounts

Data:

- Input Data: Two or more test user accounts; two distinct target clans; at least one Public Clan and one Private Clan
- Input Commands: Join Clan A; attempt to join Clan B while still a member of Clan A; Leave Clan A; owner removes a member; owner transfers ownership; request to join a Public Clan; attempt to join a Private clan without an invitation
- Output Data: Updated membership Records and roles
- System Messages: Rejection message when a user attempts a second active membership; rejection message when a user attempts to join a Private Clan without an invitation

Procedures:

1. User Joins Clan A; confirm membership is recorded with role Member
2. The same user attempts to join Clan B without leaving Clan A ; confirm the system rejects this
3. User leaves Clan A; confirm membership is removed and the user can now join Clan B
4. Clan Owner removes a member from their clan; confirm the membership is revoked
5. Clan owner transfers ownership to another member; confirm roles update accordingly
6. A user request to join a Public Clan without an invitation; confirm the request is accepted per the platform's public join rules
7. A user request to join a Private Clan without an invitation; confirm the system reject this, since private clans is invitation only

Test Analysis Report

To be completed following test execution in Sprint 2/3 . Planned analysis criteria:

- **Function:** Confirm all membership operations, the single active membership constraint, and correct enforcement of Public vs Private join rules
- **Performance:** Pass if join/ leave /remove/transfer actions each completed within 2 sec; fail if any action exceeds this threshold
- **Data measures:** Pass/ Fail per operation (7 checks in Total)
- **Errors/Deficiencies:** Any breach of the one active membership rule is high severity and should be logged with priority

---

Test D: Invitation Workflow

Verifies the full invitation lifecycle, including its status state machine (Pending, Accepted, Declined, Expired, Revoked)

Test Specification

Demonstrates satisfaction of the client's Invitation System requirements: invite, list invitations, accept, decline and correct creation/expiration timestamps and status tracking.

Test Description

A test description consists of:

Location of test: Web UI invitation pages and the corresponding REST endpoints (invite, list invitations, accept, decline)

Means of Control: Manual, using an owner/ moderator test account and an invitee test account

Data:

- Input Data: Invitee user ID and target clan ID
- Input Commands: Send an invitation; view pending invitations; accept an invitation; decline an invitation; revoke an invitation before it is answered
- Output Data: An invitation record with creation timestamp, expiration timestamp and status
- System Messages: Confirmation of send/accept/decline/revoke; an error if reposting to an already resolved or expired invitation

Procedures:

1. Owner/moderator invites a user ; confirm the invitation is created with status Pending and Correct timestamps.
2. Invitee views their pending invitations; confirm the invitation is visible.
3. Invitee accepts the invitation; confirm status changed to Accepted and a membership is created.
4. Repeat with a second invitation; owner revokes it before a response; confirm status changes to Revoked and it can no longer be accepted
5. Repeat with a third invitation; invitee declines; confirm status changes to Declined
6. Confirm an invitation past its expiration timestamp and can no longer be accepted (Expired)

Test Analysis Report

To be completed following test execution in Sprint 2/3 . Planned analysis criteria:

- **Function:** Confirm invite/revoke/view/accept/decline behavior and correct state transition
- **Performance:** Pass if invite/accept/decline/revoke actions each complete within 2 seconds; fail if any action exceeds this threshold.
- **Data measures:** Pass/Failed per state transition (Pending - Accepted, Declined - Revoked - Expired)
- **Errors/Deficiencies:** Any incorrect state transition, or a resolved invitation that can still be actioned, should be logged as high severity

---

Test E: Admin Moderation /Interface

Verifies administrator capabilities to list, search, suspend, rename, delete clans, and review user reports and moderation history.

Test Specification

Demonstrates satisfaction of the client's Administration Interface requirements, including clan moderation actions, the report queue, and access control restricting these actions to administrators.

Test Description

Location of test: Admin Web interface and the admin REST endpoints (list clans, list reports)

Means of Control: Manual, using an admin test account and a standard user test account

Data:

- Input Data: Existing clan records; a user submitted report (eg. offensive tag)
- Input Commands: Submit a report as a standard user; as admin ; list/ search clans, view clan details, suspend a clan, rename a clan, delete clan, review the report and moderation history
- Output Data: Updated clan status (eg. suspended/renamed/deleted); the report visible in the admin queue; a moderation history entry
- System Messages: Confirmation of each admin action; an access denied message if a non admin attempts these action

Procedures:

1. Standard user submits a report against a clan for an offensive tag; confirm it appears in the admin reports queue.
2. Admin reviews the report and the clan's moderation history
3. Admin suspends the clan; confirm the suspended status is visible and an audit history event is recorded
4. Admin renames the clan; confirm the update and corresponding audit log entry
5. Admin deletes a clan; confirm the update and corresponding audit log entry
6. Admin modifies a clan's invalid tag to a valid one; confirm the update is successful and logged
7. Confirm a non admin user cannot access any of the admin actions

Test Analysis Report

To be completed following test execution in Sprint 2/3 . Planned analysis criteria:

- **Function:** Confirms admin moderation actions, report queue and the access control
- **Performance:** Pass if suspend/rename/delete/tag modification actions each complete within 2 seconds; fail if any action exceeds this threshold
- **Data measures:** Pass/Fail per action (7 checks), plus confirmation each generates the correct audit history event
- **Errors/Deficiencies:** Any admin action reachable by a non admin user is a critical security defect

---

Test F: REST API

Verifies that the document REST API works per its OpenAPI specification for the core clan endpoints

Test Specification

Demonstrates satisfaction of the client's REST API requirements: correct request/ response schemas, HTTP status codes, and error handling for the clan CRUD endpoints.

Test Description

Location of test: REST API, tested directly

Means of Control: Automated, via a scripted API test suite

Data:

- Input Data: A valid clan-creation payload; an invalid payload missing required fields; a valid clan ID; a non existent clan ID
- Input Commands: POST/clans (valid and invalid payloads); GET /clans; GET /clans/{id} (valid or non existent IDs) ; PATCH /clans/{id}; DELETE /clans/{id}
- Output Data: Json responses matching the documented OpenAPI schema, with correct HTTP status codes
- System Messages: JSON error bodies describing the failure for invalid requests

Procedures:

1. POST a valid clan payload; confirm a 201 response matching the documented schema
2. POST an invalid payload; confirm a 400 response with a descriptive error body
3. GET /clans; confirm a 200 response including the newly created clan
4. GET /clans/{id} for the created clan (200,correct data) and for a non existent ID (404)
5. Patch /clans/{id}; confirm the update is applied and reflected on a subsequent GET
6. DELETE /clans/{id}; confirm success and that a subsequent GET returns 404

Test Analysis Report

To be completed following test execution in Sprint 2/3 . Planned analysis criteria:

- **Function:** Confirms the API's CRUD behaviour and error handling match the OpenAPI contract
- **Performance:** Record baseline response times for each call
- **Data measures:** Pass/Fail per endpoint/status code combination (6 checks), plus schema validation
- **Errors/Deficiencies:** Any mismatch between actual and documented API behavior should be logged and corrected before next client demo

---

Testing Schedule

The specific testing arrangements will be confirmed before the relevant tests are executed in Sprint 2 and Sprint 3.

Test	*Dates*	*Responsible*	*Dependencies*
Test A	*Sprint 2*	*TBD*	*OIDC integration and protected routes implemented*
Test B	*Sprint 2*	*TBD*	*Clan creation UI/API and tag validation implemented*
Test C	*Sprint 3*	*TBD*	*Membership operations and role management implemented*
Test D	*Sprint 3*	*TBD*	*Invitation system implemented*
Test E	*Sprint 3*	*TBD*	*Administration interface, reporting and moderation functions implemented*
Test F	*Sprint 2, 3*	*TBD*	*Core REST API endpoints and OpenAPI documentation implemented*

Testing Materials

Hardware

A computer capable of running Clan Management Platform and supporting services.

Software

- *Supported web browser*
- *Clan Management Platform test build*
- *PostgreSQL*
- *Docker*
- *External OIDC test provider*
- *API testing tool*

Data Sets

- *Public and private clans*
- *Valid and invalid clan tags*
- *Test membership records*
- *Test invitations*
- *Test reports and moderation records*

Test Accounts

- *Standard authenticated user*
- *Clan owner*
- *Moderator*
- *Clan member*
- *User with no active clan membership*
- *Platform administrator*

Project Resources

- *Client Project Requirements Specification*
- *Scope of Work*
- *Sprint 2 User Stories*
- *FAF Github repositories* and *API documentation*